Module 2: Intrinsically Interpretable Models

Instructional Hours: 6

Module Overview

In this module, you will learn about techniques used to make transparent the decision making process of models that are classified as intrinsically interpretable. It is usually possible for humans to understand how such models go about their decision making process. You will implement python code that explains the output of a linear regression model.

Learning Outcomes

At the end of this module, you will be able to:

1. Identify some of the intrinsically interpretable Al models
2. Discuss the techniques used to explain intrinsically interpretable Al models
3. Implement code to create an Al model that include explainability
4. Interpret intrinsically interpretable model outputs to key stakeholders

Learning Activities/Task List

1. Review Introductory notes on intrinsically interpretable AI models
2. Contribute to a discussion on intrinsically interpretable AI models
3. Review Introductory notes on Explaining Linear Regression Models
4. Implement Python Code to Explain a Linear Regression Model
5. Review extra reading materials on Intrinsically interpretable models
6. Contribute to a discussion on one intrinsically interpretable Al models
7. Complete a final quiz on intrinsically interpretable Al models

Overview Of Intrinsically Interpretable Models

As discussed in the previous module, Explainable AI (XAI) objective is to make the decision-making processes of Al models transparent and understandable to humans. Al models are becoming increasingly complex which has resulted in the demand for models not only to perform well but also to be interpretable. Intrinsically interpretable models are those who's internal decision process and

predictions can be easily understood by humans. These models are inherently understandable without requiring additional tools or techniques. In the next section, we will discuss some of these models.

Linear regression and logistic regression

The relationship between input features and output is linear and the effect of the coefficients can be determined easily. The coefficients directly indicate the strength and direction of the relationship between each feature and the output variable. In a linear regression model, a positive coefficient for an attribute implies that an increase in that feature will result in an increase the predicted outcome. The coefficient represents the change in the output variable that will be observed if there is a unit change in its corresponding attribute with all other attributes remaining constant. The same applies to logistic regression except that a coefficient value represents the change in the log of the odds ratio if there is a unit change in the corresponding variable where the odds is defined as $\mathrm{P(x)}/(1-\mathrm{P(x)})$. The actual change in odds given the coefficient value k, is given by the formula e^k.

Decision Trees

Classification and Regression Trees (CART) provide comprehensible treestructured representation of the decision path for any sample input. It is based on simple yes/no questions at each node of the tree. For CART trees, if the value of the attribute at the node satisfies the yes cut-off criteria, the yes path is followed, else the no path is followed.
The path from the root to a leaf node in the tree represents a decision rule, which can be easily interpreted. All the data samples that end up in the same leaf node of a tree are classified in the same way or have the same regression value. The regression value is the average of the samples at the
lead node. The classification values is the class of the majority of the (training) samples at the leaf node. If-then rules can be used to represent the decision criteria for each leaf node.

Features importance is another technique that can be used to make transparent the decision making criteria of decision trees. An attribute's importance is calculated by getting the sum of its reduction of the variance/Gini index of all it's the parent nodes. The higher the total reduction, the more important the attribute is. This can help in explaining which attributes have the most weight in the CART tree decision making.

Rule-Based Models

These include decision rules and association rule models which operate using if-then logic, which is easy to understand. The algorithms generate human-readable rules that represent the logic behind the predictions.
RuleFit is an example of an algorithm that generates rules using decision tree

based methods. The rules are used to capture interactions between the attributes (Molnar C., 2022). The rules are then used as attributes in linear regression based models. The regression model consists of both the original attributes and the rules. The coefficient of both the rules and original attributes determine their importance. The same techniques used to explain linear regression models can be used to explain RuleFit models since in the end, it results in a linear regression model.
One R is another algorithm that learns rules from one feature which means that only one attribute and its possible categories is used to classify instances (Molnar C., 2022). As an example, assume that dataset on house prices has attributes such as location, size, and number of rooms. You can use an attribute such as location (good, medium, bad) to decide the price (high, low). If most houses in a bad location have low prices then we will have the rule (IF location=bad then price=low). If most houses in a medium location have high prices then we will have the rule, (IF location=medium then price=high). Other attributes will also be used to create their own set of rules. Only the attribute whose rules result in the fewest errors is used by the OneR method. Other algorithms for creating decision rules are Sequential Covering and Bayesian Rule list.

As earlier stated, decision rules are explainable by default because they are human readable. Two other metrics used to explain decision rules are support (also called coverage) and Accuracy (also called confidence of a rule).
Support is the percentage of the data instances to which the antecedent (condition) part of the rule applies. For instance, if you have the rule (IF location=bad then price=low), then, the coverage of the rule is the percentage of data instances for which this IF condition is true. The

Accuracy of a rule is the percentage of the number of instances that the rule covers that it predicts correctly.

Linear Regression Explanation Methods

In this section, you will review the methods used to explain linear regression models. You will see an example used to illustrate the use of these techniques. A linear regression model has the form

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

β_i is the coefficient of attribute x_i. β_i is known as the intercept and is the value of $\hat{y}$ when all the attributes have a value of zero. The interpretation given to the other coefficients is as explained below.
If x_i is a numerical feature, increasing its value by one increases the value of $\hat{y}$ by β_i provided that all the other attributes remain constant.

If x_i is a binary feature (0 or 1), if we take 0 as the reference value, the value of $\hat{y}$ increases by Θ_i if x_i is changed from zero to one with all other attributes remaining constant.

If x_i is a categorical feature, one hot encoding can be used to change it to several binary features. The interpretation of the binary features will then apply.

The other measure used to interpret linear regression models is features importance. The feature importance is calculated using the formula below:

$$\text{importance}\left(\beta_i\right) = \frac{\beta_i}{\text{SE}\left(\beta_i\right)}$$

Where $SE\left(\beta_i\right)$ is the standard error of coefficient. A coefficient with a higher standard error has its importance reduced because of its lack of certainty. The importance of an attributes can be plotted on with a horizontal line representing each coefficient. The width of the line represents the coefficient of the attribute and its confidence interval. The center of the line represents the mean value of the attribute. The vertical position of the line represents its importance with more important attributes being higher in the graph.

The R^2 score is another parameter used to explain the decision criteria of a linear regression model. Its range should be from 0 to 1 where 1 means that the model entirely explains the variance in the output variable. However its value can be less than one which means the model does not at all explain the variance in the output variable. It is given by the formula below:

$$R^2 = \sum_{i=1}^{n} \frac{\left(y_i - \hat{y}_i\right)^2}{\left(y_i - \bar{y}\right)^2}$$

$\mathrm{y_i}$ is the actual output for example $\mathrm{i}, \hat{y}_i$ is the predicted value and $\bar{y}$ is the mean of the output value.

Implementation Of Linear Regression Explainability Using Python

In this example, we are going to use the diabetes dataset from the scikitlearn Python package to calculate the value of coefficients and explain the resulting model. The code is given below:

```
%{language=python}
#import the necessary packages
import statsmodels.api as sm
import statsmodels.formula.api as smf
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error as mse, r2_score
from sklearn.datasets import load_diabetes
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
#Load the data
data=load_diabetes()
#print(data.DESCR)
#store the predictors in and target in separate variables
x=data.data
y=data.target

print(x.shape)
print(y.shape)
# Prepare the data for the StatsModel module by adding a constant.
#One more attribute whose values is zero will be added to each row
x=sm.add_constant(x)
print(x.shape)
#Get the columns of the data
columns=data.feature_names.copy()
columns.insert(0, 'constant')
print(columns)
#Put the data in a dataframe for easier visualization
import pandas as pd
df = pd.DataFrame(x, columns=columns)
df['target'] = y
df.head()
#show the correlation between the variables using a correlation matrix
corr_matrix = df.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",
linewidths=0.5)
# Add a title
plt.title('Correlation Heatmap')
plt.show()
# split the data into training and test set
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.4, random_state=42)
# fit the model
model = sm.OLS(y_train, x_train).fit()
print(model.summary())
#Visualise some of the predictions and compare them with the target
values
predictions=model.predict(x_test)
for val in predictions[0:10]:
    print("{:.0f}\t".format(val), end="")
print()
for val in y_test[0:10]:

    print("{:.0f}\t".format(val), end="")
```

```
#view RMSE values and R-Squared values
import numpy as np
rms=np.sqrt(mse(y_test,predictions))
print("RMSE: {:.2f}".format(rms))
print("R2: {:.2f}".format(model.rsquared))
print("Adjusted R2: {:.2f}".format(model.rsquared_adj))
#store the coefficents and their confidence intervals in
#a dataframe for easier visualization. Also calculate the importance
of each
#attribute
std_errors = model.bse
#95% confidence intervals for the coefficients
conf_intervs = model.conf_int()
#create a dataframe
# Combine into a DataFrame for easy viewing
print(conf_intervs[:,0])
print(conf_intervs[:,1])
dfSummary = pd.DataFrame({
    'Variable': columns,
    'Coefficient': model.params, #Coefficient value
    'Standard Error': std_errors,
    'coeff_lb': conf_intervs[:,0],#lower bounds of the interval
    'coeff_ub': conf_intervs[:,1]#upper bounds of the interval
})
#lb and ub confidence interval lower bound and upper bound
respectively
dfSummary["importance"]=dfSummary["Coefficient"]/dfSummary["Standard
Error"]
dfSummary.head(11)
#Sort the values
dfSorted=dfSummary.sort_values(by="importance", ascending=False)
dfSorted.head(11)
#remove the intercept coefficient and its other associated information
lineLabels=dfSorted['Variable'].values[1:-1]
lowerBounds=dfSorted['coeff_lb'].values[1:-1]
upperBounds=dfSorted['coeff_ub'].values[1:-1]
coefficients=dfSorted['Coefficient'].values[1:-1]

importances=dfSorted['importance'].values[1:-1]
#get the minimum and the maximum of the coefficients' upper and lower
bounds
#This will help in setting the dimensions of the graph
#and generating the axis labels
temp=np.array(lowerBounds)
minX=temp.min()
temp=np.array(upperBounds)
```

```
maxX=temp.max()
xvalues=np.linspace(minX, maxX, 10 , dtype='int')
temp=np.array(importances)
maxX=int(temp.max()+1)
minX=int(temp.min()-1)
yvalues=np.linspace(minX, maxX, 10, dtype='int')
#remove the intercept coefficient and its other associated information
lineLabels=dfSorted['Variable'].values[1:-1]
lowerBounds=dfSorted['coeff_lb'].values[1:-1]
upperBounds=dfSorted['coeff_ub'].values[1:-1]
coefficients=dfSorted['Coefficient'].values[1:-1]
importances=dfSorted['importance'].values[1:-1]
#get the minimum and the maximum of the coefficients' upper and lower
bounds
#This will help in setting the dimensions of the graph
#and generating the axis labels
temp=np.array(lowerBounds)
minX=temp.min()
temp=np.array(upperBounds)
maxX=temp.max()
xvalues=np.linspace(minX, maxX, 10 , dtype='int')
temp=np.array(importances)
maxX=int(temp.max()+1)
minX=int(temp.min()-1)
yvalues=np.linspace(minX, maxX, 10, dtype='int')
#create the points for the horizontal/importance lines of the
coefficients
#also a mide vertival line to show the actual coefficient values
left=[] #stores left points of importance lines
right=[] #stores right points of importance lines
top=[] #stores top point of vertical line marking cefficient location
bottom=[] #stores bottom point of vertical line marking cefficient
location

for x,y,z,w in zip(importances, lowerBounds, upperBounds,
coefficients):
    left.append([y,z])
    right.append([x,x])
    top.append([w,w])
    bottom.append([x-0.02,x+0.02])
#Plot the lines
fig, ax = plt.subplots()
ax.figure.set_figwidth(6)
ax.figure.set_figheight(6)
for i in range(len(left)):
    ax.plot(left[i], right[i], linestyle='-', label=lineLabels[i])
```

```
    ax.plot(top[i], bottom[i], color='red')
ax.set_xticks(xvalues,xvalues)
ax.set_yticks(yvalues,yvalues)
ax.legend()
```

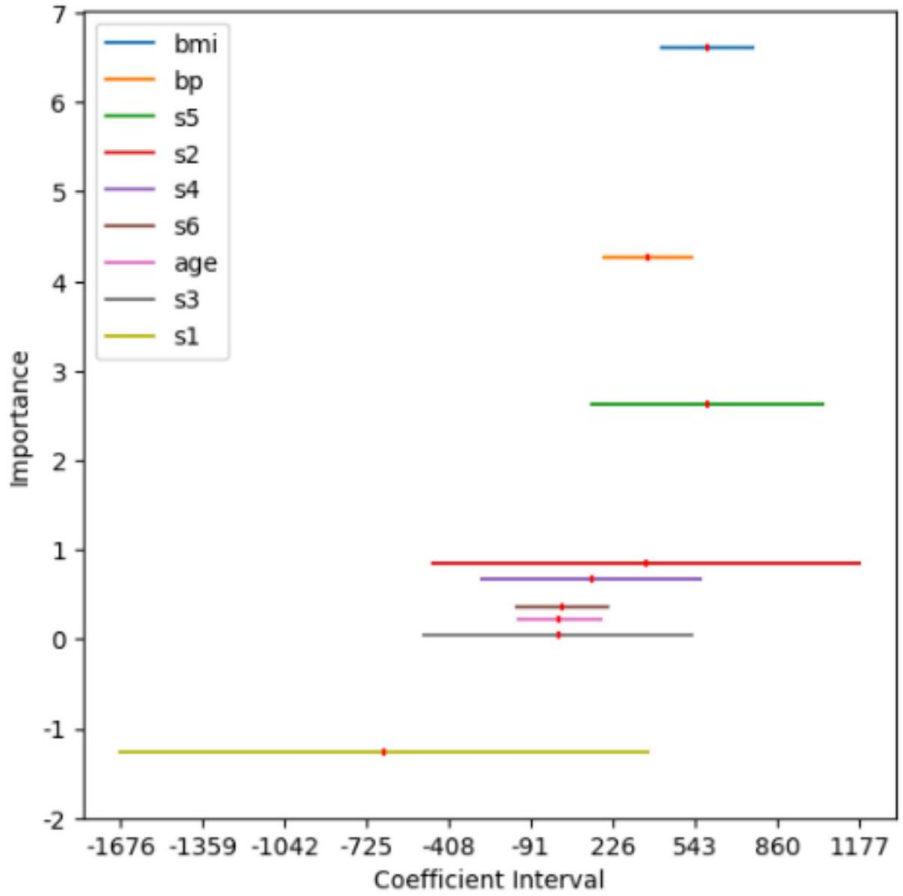

Figure 2.2: A plot of Feature importance

From figure 2.2, we can see that the bmi is the most important feature in prediction. This can be confirmed by the correlation plot in Figure 2.3.

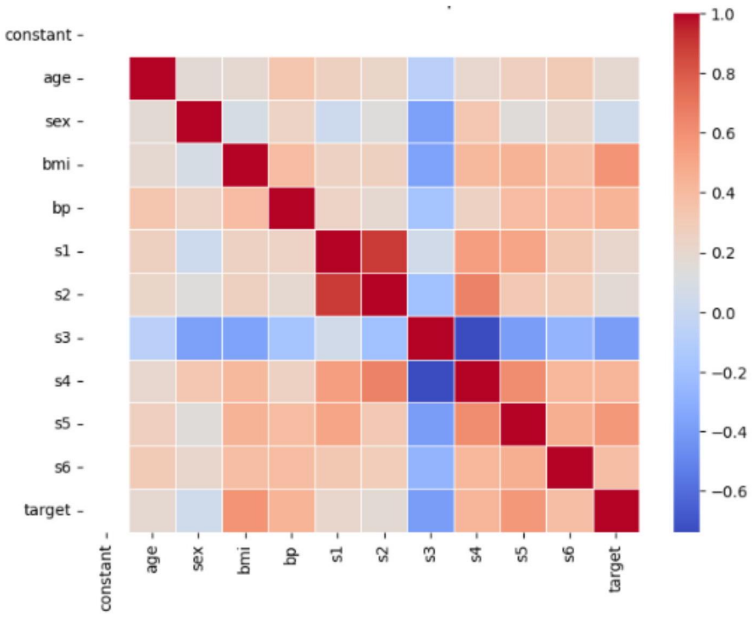

Figure 2.3: Correlation heatmap for the diabetes datase †

Reading Material 1

To learn more about intrinsically interpretable models read the following material

Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.), url: **https://christophm.github.io/interpretable-ml-book/simple.html**